

Busca em árvore de Monte Carlo e a família AlphaZero

Busca baseada em simulação • UCT • AlphaGo, AlphaZero e MuZero

Prof. Marcelo Keese Albertini • Faculdade de Computação

Adaptado e expandido de CS234: *Reinforcement Learning*, Emma Brunskill, Stanford University — Lecture 13

Intuição

Todo o resto do curso constrói objetos *globais*: uma tabela V^* , uma rede π_θ . Esta aula faz a pergunta oposta — se eu tenho um simulador e alguns segundos sobrando, o que consigo comprar de qualidade de decisão *aqui, neste estado*? E, mais importante: como converter esse esforço efêmero em aprendizado permanente. A resposta a essa segunda pergunta é o algoritmo AlphaZero.

1 Planejamento em tempo de decisão

Vale separar dois usos de um modelo:

- ▶ **Planejamento em segundo plano** (*background planning*). O modelo é usado para melhorar uma política ou função valor *global*, que depois é consultada em tempo de execução. Iteração de valor e Dyna são disso.
- ▶ **Planejamento em tempo de decisão** (*decision-time planning*). O modelo é usado *no momento de agir*, a partir do estado corrente, e o resultado da computação é descartado logo depois. MCTS é disso.

O segundo é atraente quando $|S|$ é grande demais para qualquer objeto global ser bom em todo lugar, mas o agente visita, em uma partida, uma fração minúscula dos estados. Concentrar o esforço nessa fração é uma economia gigantesca.

Definição — Modelo generativo

Uma caixa-preta $\mathcal{M}$ que, dado (s, a) , devolve amostras $s' \sim P(\cdot \mid s, a)$ e $r \sim R(\cdot \mid s, a)$. Não exige acesso às probabilidades, nem enumeração de sucessores. Em jogos de tabuleiro, o modelo são as regras: perfeito e barato.

Atenção

Todo o método depende dessa hipótese. Em domínios com modelo aprendido, o erro por passo se compõe ao longo do horizonte simulado: um simulador com 1% de erro por passo é praticamente ruído após cinquenta passos. É por isso que MCTS brilhou em jogos e continua difícil em robótica de contato ou em saúde.

2 Busca de Monte Carlo simples

Algoritmo — Busca de Monte Carlo simples

Dados $\mathcal{M}$, uma política de simulação π e um orçamento K por ação:

1. para cada $a \in \mathcal{A}$, simule K episódios a partir do estado real s_t começando com a ação a e seguindo π depois;
2. estime $\hat{Q}(s_t, a) = \frac{1}{K} \sum_{k=1}^K G_t^k$;
3. execute $a_t = \arg \max_a \hat{Q}(s_t, a)$.

Proposição — um passo de melhoria

Seja π' a política que joga $\arg \max_a Q^\pi(s_t, a)$ em s_t e segue π daí em diante. Então $V^{\pi'}(s_t) \geq V^\pi(s_t)$, e a busca de Monte Carlo simples converge para π' quando $K \rightarrow \infty$.

Demonstração

Por definição, $\hat{Q}(s_t, a) \rightarrow Q^\pi(s_t, a)$ pela lei dos grandes números, pois cada episódio é uma amostra independente do retorno sob π após a ação a . Logo o $\arg \max$ empírico converge para o $\arg \max$ verdadeiro (assumindo unicidade). E

$$V^{\pi'}(s_t) = \max_a Q^\pi(s_t, a) \geq \sum_a \pi(a | s_t) Q^\pi(s_t, a) = V^\pi(s_t),$$

porque o máximo domina qualquer média. □

Duas consequências práticas. Primeiro, o **teto**: por melhor que seja a amostragem, o resultado nunca supera π' — um único passo de melhoria sobre a política de simulação. Segundo, o **ruído**: $\hat{Q}$ é média de retornos, cuja variância cresce com o horizonte. Tomar o $\arg \max$ de estimativas ruidosas produz *viés de maximização*: em expectativa, $\mathbb{E}[\max_a \hat{Q}] \geq \max_a \mathbb{E}[\hat{Q}]$, isto é, superestimamos sistematicamente o valor da ação escolhida — o mesmo fenômeno que motivou o *double Q-learning* na Aula 4.

2.1 A árvore expectimax e por que ela não serve

A alternativa ideal é maximizar em *todos* os níveis, não só no primeiro:

$$Q_h(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q_{h-1}(s', a'), \quad Q_0 \equiv 0,$$

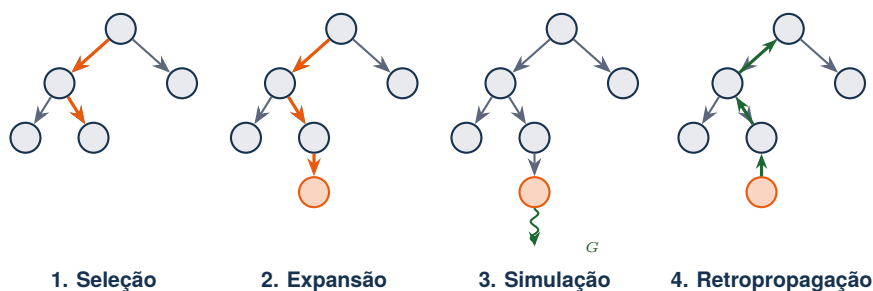
avaliado por busca dirigida para frente a partir de s_t — resolver apenas o *sub-MDP* que começa agora, e não o MDP inteiro. O problema é o tamanho: uma árvore completa de profundidade H tem $(|S||\mathcal{A}|)^H$ nós. Para o *Go*, com ramificação $b \approx 250$ e partidas de $d \approx 150$ jogadas, isso está fora de cogitação, e a poda alfa-beta — que exige minimax determinístico e uma boa função de avaliação escrita à mão — apenas divide o expoente por dois.

Intuição

MCTS ataca as duas maldições ao mesmo tempo: **amostra** em vez de somar sobre s' (largura) e **trunca** a subárvore abaixo de uma folha, substituindo-a por uma estimativa de valor (profundidade).

3 Busca em árvore de Monte Carlo

3.1 As quatro fases



1. **Seleção.** A partir da raiz, desça pela árvore escolhendo ações com a *política de árvore* (UCT ou PUCT), que depende das estatísticas acumuladas. Pare ao chegar a um nó com alguma ação nunca tentada, ou a um estado terminal.
2. **Expansão.** Acrescente à árvore o nó alcançado.
3. **Simulação.** Estime o valor da nova folha. Ou por *rollout* — jogar até o fim com uma política fixa e barata — ou por uma rede de valor $v_\theta(s_{\text{folha}})$.
4. **Retropropagação.** Percorra o caminho de volta até a raiz atualizando $N(s, a)$, $W(s, a)$ e $\hat{Q}(s, a)$.

A separação entre a *política de árvore* (adaptativa, cara, informada) e a *política de rollout* (fixa, barata, ignorante) é o que dá ao algoritmo o seu perfil: ele investe onde já sabe alguma coisa e economiza onde não sabe.

3.2 Estatísticas mantidas

Estatística	Significado	Atualização
$N(i, a)$	vezes que a foi escolhida no nó i	+1 por visita
$W(i, a)$	soma dos retornos observados	+ G
$\hat{Q}(i, a)$	valor médio $W(i, a)/N(i, a)$	recalculado
$N(i)$	$\sum_a N(i, a)$	+1

A memória cresce com o número de nós *visitados*, não com $|S|$. É esse detalhe — e não a esperteza da regra de seleção — que torna o método aplicável a espaços de estados astronômicos.

3.3 UCT

Dentro da árvore, cada nó é um problema de bandit: os braços são as ações, o ganho é o retorno da simulação que passa por ali. Aplicar UCB1 em cada nó dá o **UCT**.

Regra de seleção UCT (Kocsis e Szepesvári, 2006)

$$a = \arg \max_{a \in \mathcal{A}(i)} \underbrace{\hat{Q}(i, a)}_{\text{aproveitamento}} + c \underbrace{\sqrt{\frac{\ln N(i)}{N(i, a)}}}_{\text{exploração}}, \quad \hat{Q}(i, a) = \frac{1}{N(i, a)} \sum_{k=1}^{N(i, a)} G^k(i, a).$$

Ações com $N(i, a) = 0$ recebem bônus infinito e são visitadas primeiro.

Três observações que costumam ser fonte de erro:

- ▶ o bônus decai como $N(i, a)^{-1/2}$ e cresce apenas como $\sqrt{\ln N(i)}$ — dobrar o orçamento total quase não muda o equilíbrio, o que faz a busca *concentrar* esforço;
- ▶ c só tem significado se os retornos estiverem em uma escala conhecida. Normalize G para $[0, 1]$ (ou $[-1, 1]$ em jogos de soma zero);
- ▶ como as estatísticas mudam a cada simulação, a política de árvore muda junto: o algoritmo faz melhoria de política *durante* a busca, e não apenas ao final.

Algoritmo — UCT

entrada: s_t , modelo $\mathcal{M}$, orçamento K , constante c
 árvore $\leftarrow$ raiz $s_{\text{raiz}} = s_t$
para $k = 1, \dots, K$:
 $s \leftarrow s_{\text{raiz}}$; caminho $\leftarrow []$
 enquanto s está na árvore e todas as ações de s já foram tentadas:
 $a \leftarrow \arg \max_a \hat{Q}(s, a) + c\sqrt{\ln N(s)/N(s, a)}$
 $(s', r) \sim \mathcal{M}(s, a)$; anexe (s, a, r) ao caminho; $s \leftarrow s'$
 se s não é terminal: acrescente s à árvore
 $G \leftarrow \text{AVALIA}(s)$
 para (s, a, r) no caminho, de trás para frente:
 $G \leftarrow r + \gamma G$; $N(s, a) += 1$; $W(s, a) += G$; $\hat{Q}(s, a) \leftarrow W(s, a)/N(s, a)$
devolva $\arg \max_a N(s_{\text{raiz}}, a)$

Intuição

Por que devolver a contagem de visitas, e não $\hat{Q}$? Porque $N(s_{\text{raiz}}, a)$ é uma estatística acumulada sobre toda a busca: uma ação só recebe muitas visitas se pareceu boa *de forma persistente*. Já $\hat{Q}(s_{\text{raiz}}, a)$ pode ser inflado por um único retorno afortunado em um ramo pouco visitado — exatamente o caso em que a variância é maior.

3.4 O que se pode provar — e o que não

Consistência do UCT

Sob retornos limitados e exploração suficiente, a probabilidade de o UCT selecionar uma ação subótima na raiz tende a zero quando o número de simulações $K \rightarrow \infty$, e a estimativa na raiz converge para o valor ótimo do sub-MDP.

Atenção

A garantia é assintótica e o pior caso é severo: Coquelin e Munos (2007) exibem problemas em que a convergência do UCT é *duplamente exponencial na profundidade*. A intuição é simples — o bônus logarítmico do UCB pressupõe que as recompensas de um braço são estacionárias, o que não vale dentro de uma árvore, onde o valor de um ramo muda à medida que a subárvore é explorada. Domínios com uma única sequência estreita de jogadas boas (“agulha no palheiro”) derrotam o algoritmo.

Na prática o método funciona por dois motivos que a teoria de pior caso não captura: os domínios costumam ter muitos caminhos razoáveis em vez de um só, e — sobretudo — um bom *prior* de política reduz drasticamente a ramificação efetiva. É esse segundo ponto que a seção seguinte explora.

3.5 Vantagens, limitações e remédios

Vantagens

- ▶ busca best-first altamente seletiva;
- ▶ avalia dinamicamente o estado atual;
- ▶ amostragem quebra a maldição da dimensão;
- ▶ funciona com modelo caixa-preta;
- ▶ *anytime* e paralelizável;
- ▶ dispensa heurística de domínio;
- ▶ mais computação $\Rightarrow$ melhor decisão, sem retreinar.

Limitações e remédios

- ▶ ramificação alta $\rightarrow$ *prior* de política (PUCT);
- ▶ ações contínuas $\rightarrow$ *progressive widening*;
- ▶ *rollouts* fracos $\rightarrow$ rede de valor v_θ ;
- ▶ poucos dados por nó $\rightarrow$ RAVE/AMAF, transposições;
- ▶ modelo imperfeito $\rightarrow$ horizonte curto, ou modelo aprendido no espaço de valores (MuZero);
- ▶ recompensa esparsa $\rightarrow$ horizonte maior ou *shaping*.

4 De AlphaGo a MuZero

4.1 AlphaGo (2016)

Três redes e uma busca: uma rede de política treinada por aprendizado supervisionado sobre partidas humanas (p_σ), uma rede de política refinada por auto-jogo (p_ρ), e uma rede de valor v_θ treinada sobre posições de auto-jogo. A busca usava p_σ como *prior* e avaliava folhas por uma mistura

$$\hat{V}(s_{\text{folha}}) = (1 - \lambda) v_\theta(s_{\text{folha}}) + \lambda z_{\text{rollout}},$$

combinando duas fontes com erros de natureza diferente. Um detalhe contraintuitivo e instrutivo: a rede *supervisionada* funcionava melhor como *prior* da busca do que a rede refinada por RL — porque é mais *diversa*, e uma busca precisa de diversidade para não colapsar.

4.2 AlphaGo Zero (2017): remoção sucessiva

	AlphaGo	AlphaGo Zero
Partidas humanas	sim, 30M posições	não
Features de domínio	sim, dezenas	só o tabuleiro
Rollouts na busca	sim	não
Redes	três	uma, duas cabeças
Arquitetura	convolucional	residual

A rede única produz $(p, v) = f_\theta(s)$, com $p \in \Delta(\mathcal{A})$ o *prior* de política e $v \in [-1, 1]$ o valor da posição do ponto de vista do jogador da vez.

Regra de seleção PUCT

$$a = \arg \max_a Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}, \quad P(s, a) = p_a.$$

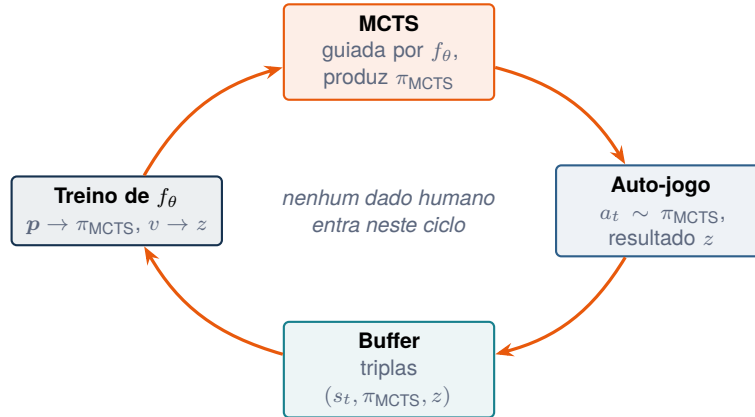
Comparada ao UCT: o bônus é *multiplicado* pelo *prior*, de modo que ações implausíveis para a rede praticamente não são exploradas; o decaimento $1/(1 + N(s, a))$ é mais agressivo que $1/\sqrt{N(s, a)}$; e não há logaritmo. Com $b \approx 250$, é o *prior* que torna a busca viável — ela examina a sério meia dúzia de jogadas em vez de duzentas e cinquenta. A rede não substitui a busca: ela a poda.

4.3 A ideia central: a busca é um operador de melhoria de política

Ao fim de K simulações, defina a **política da busca** na raiz:

$$\pi_{\text{MCTS}}(a \mid s) = \frac{N(s, a)^{1/\tau}}{\sum_b N(s, b)^{1/\tau}}, \quad \tau > 0.$$

Empiricamente, π_{MCTS} joga melhor que o *prior* p que a gerou — a busca gastou computação e converteu isso em qualidade. AlphaZero fecha o ciclo tratando π_{MCTS} como *alvo de treinamento* para p , e o resultado da partida z como alvo para v .



Perda do AlphaGo Zero / AlphaZero

Para cada posição s do auto-jogo, com alvos $\pi = \pi_{\text{MCTS}}(\cdot \mid s)$ e $z \in \{-1, 0, +1\}$:

$$\ell(\theta) = \underbrace{(z - v_\theta(s))^2}_{\text{regressão de valor}} - \underbrace{\pi^\top \log p_\theta(s)}_{\text{destilação da busca}} + \underbrace{c \|\theta\|^2}_{\text{regularização}}.$$

Esta é a leitura que vale guardar: **é iteração de política**, com a busca no papel do passo de melhoria e o auto-jogo no papel da avaliação. Anthony, Tian e Barber (2017) batizaram o esquema de *expert iteration* — a busca é o “especialista lento”, a rede é o “aprendiz rápido” que *amortiza* o especialista, tornando a próxima busca mais barata e mais afiada. Note também o que *não* aparece na perda: nenhum gradiente de política, nenhuma razão de importância, nenhuma região de confiança. Todo o problema difícil de RL foi empurrado para dentro da busca, e o que sobrou é aprendizado supervisionado.

4.4 Duas engenhocas indispensáveis

Definição — Ruído de Dirichlet na raiz

$P(s_{\text{raiz}}, a) = (1 - \varepsilon) p_a + \varepsilon \eta_a$, com $\eta \sim \text{Dir}(\alpha)$ e $\varepsilon \approx 0,25$. Garante que toda jogada legal tenha alguma chance de ser explorada, mesmo que a rede a considere impossível.

Definição — Temperatura

Nas primeiras jogadas de cada partida usa-se $\tau = 1$ (amostragem proporcional às visitas), o que gera diversidade de aberturas; depois $\tau \rightarrow 0$ (praticamente $\arg \max$), o que garante jogo forte.

As duas são a versão de auto-jogo do problema de exploração das Aulas 10–12. Sem elas, o sistema converge para um repertório estreito e passa a treinar contra um adversário degenerado — o auto-jogo perde a propriedade que o torna valioso, que é gerar automaticamente um currículo de dificuldade crescente.

4.5 AlphaZero e MuZero

AlphaZero (2018) aplicou a mesma receita, sem conhecimento de domínio além das regras, a Go, xadrez e shogi, superando os melhores programas especializados de cada um. Avaliava ordens de grandeza *menos* posições por jogada que os motores clássicos de xadrez — evidência direta de que um bom julgamento posicional vale por muitas simulações.

MuZero (2020) removeu a última peça de conhecimento humano: as regras. Aprende três funções — representação h_θ (observação $\rightarrow$ estado latente), dinâmica $g_\theta ((z, a) \rightarrow (z', \hat{r}))$ e predição $f_\theta (z \rightarrow (p, v))$ — e roda a busca inteiramente no espaço latente. Não há decodificador: z não precisa reconstruir o estado real, apenas predizer bem *recompensa, valor e política*, que é tudo o que o planejador consome. O modelo é aprendido *para servir ao planejamento*, e não para ser fiel.

Para discutir em aula

Se o estado latente do MuZero não precisa reconstruir o mundo, em que sentido ele ainda é um “modelo”? O que se ganha e o que se perde ao abandonar a exigência de fidelidade — e como isso se relaciona com a discussão de especificação de recompensa que veremos na próxima aula?

5 O que generaliza, e o que não

Transfere bem

- ▶ busca como operador de melhoria; rede como amortização da busca;
- ▶ *prior* de política para podar ramificação alta;
- ▶ trocar computação de decisão por qualidade de decisão;
- ▶ aprender o modelo no espaço do que o planejador consome.

Não transfere de graça

- ▶ simulador perfeito, determinístico e barato;
- ▶ recompensa terminal não ambígua (± 1);
- ▶ auto-jogo como fonte gratuita de currículo;
- ▶ episódios reinicializáveis à vontade.

O laço “gerar candidatos com um *prior*, avaliar com um verificador, destilar o resultado de volta no *prior*” reaparece hoje em modelos de linguagem que buscam sobre passos de raciocínio. A analogia é boa, mas a peça difícil migrou: em jogos, o verificador são as regras; em texto, não existe regra que declare vitória, e construir esse verificador é o problema, não a busca.

Para discutir em aula

MCTS otimiza muito bem aquilo que recebe como recompensa. Em que situações essa competência é um risco em vez de uma virtude? Traga um exemplo de um domínio em que você confiaria mais em uma política aprendida sem busca do que em uma busca profunda sobre um modelo aproximado.

6 Formulário

Busca de Monte Carlo simples

$$\hat{Q}(s_t, a) = \frac{1}{K} \sum_k G_t^k \rightarrow Q^\pi(s_t, a)$$

Backup expectimax

$$Q_h(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_{h-1}(s')$$

$$V_{h-1}(s') = \max_{a'} Q_{h-1}(s', a')$$

UCT

$$a = \arg \max_a \hat{Q}(i, a) + c \sqrt{\frac{\ln N(i)}{N(i, a)}}$$

Retropropagação

$$G \leftarrow r + \gamma G, \quad \hat{Q} \leftarrow \frac{W+G}{N+1}$$

PUCT

$$a = \arg \max_a Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}$$

Política da busca

$$\pi_{\text{MCTS}}(a | s) \propto N(s, a)^{1/\tau}$$

Perda AlphaZero

$$\ell = (z - v)^2 - \pi^\top \log p + c \|\theta\|^2$$

Ruído na raiz

$$P \leftarrow (1 - \varepsilon)p + \varepsilon \eta, \quad \eta \sim \text{Dir}(\alpha)$$

7 Exercícios

Exercício 13.1 — Aritmética do UCT

Na raiz há três ações, com as estatísticas abaixo e $N(s_{\text{raiz}}) = 100$, $c = 1$.

	a_1	a_2	a_3
$N(s_{\text{raiz}}, a)$	80	15	5
$\hat{Q}(s_{\text{raiz}}, a)$	0,62	0,55	0,70

(a) Calcule o escore UCT de cada ação e diga qual seria simulada em seguida. (b) Qual ação o algoritmo devolveria como jogada real, e por quê ela difere do item (a)? (c) Quantas visitas a_3 precisaria acumular para que seu escore caísse abaixo do de a_1 , mantendo $\hat{Q}$ fixo?

Exercício 13.2 — Viés de maximização

Sejam $\hat{Q}(a_1)$ e $\hat{Q}(a_2)$ estimativas não viesadas e independentes de dois valores verdadeiros iguais, $q_1 = q_2 = 0$, cada uma $\mathcal{N}(0, \sigma^2)$. Mostre que $\mathbb{E}[\max_a \hat{Q}(a)] = \sigma/\sqrt{\pi} > 0 = \max_a \mathbb{E}[\hat{Q}(a)]$. Explique por que isso significa que a busca de Monte Carlo simples *superestima* sistematicamente o valor da ação escolhida, e por que o efeito piora com o horizonte.

Exercício 13.3 — Visitas contra valor

Construa um exemplo numérico pequeno com duas ações na raiz em que $\arg \max_a N(s_{\text{raiz}}, a) \neq \arg \max_a \hat{Q}(s_{\text{raiz}}, a)$, com a ação de maior $\hat{Q}$ tendo apenas duas visitas. Argumente qual das duas regras você preferiria e formule a preferência em termos de erro-padrão da média.

Exercício 13.4 — PUCT e ramificação efetiva

Suponha $|\mathcal{A}| = 250$ e um *prior* p que concentra massa 0,9 em 5 ações, distribuindo o resto uniformemente. (a) Com $\sum_b N(s, b) = 800$ e $c_{\text{puct}} = 1,5$, compare o bônus PUCT de uma ação do grupo concentrado (com $N(s, a) = 100$) e de uma ação da cauda (com $N(s, a) = 0$). (b)

(continua)

Repita com p uniforme. (c) O que o resultado diz sobre a afirmação “a rede poda a busca”?

Exercício 13.5 — Alvos de treinamento

Explique por que AlphaZero usa o resultado final da partida z como alvo de valor, e não o $\hat{Q}(s_{\text{raiz}})$ produzido pela busca, que aparentemente é mais informativo. Discuta em termos de viés e variância, e do risco de o valor aprendido realimentar seus próprios erros.

Exercício 13.6 — Sinal em jogos de dois jogadores

Um aluno implementa MCTS para o jogo da velha e esquece de inverter o sinal do retorno a cada nível na retropropagação. Descreva com precisão o comportamento resultante do agente: contra que tipo de adversário ele parecerá competente, e em que momento a falha se revela?

Exercício 13.7 — Modelo imperfeito

Um simulador aprendido erra o estado seguinte com probabilidade ϵ por passo, de forma independente. (a) Qual a probabilidade de uma trajetória simulada de H passos estar inteiramente correta? (b) Para $\epsilon = 0,02$, qual o maior H com probabilidade de acerto acima de 0,5? (c) Que implicação isso tem para a escolha entre um horizonte de busca longo com *rollouts* e um horizonte curto com rede de valor?

8 Gabarito comentado (13.1 e 13.2)

13.1. Com $\ln 100 \approx 4,605$: o bônus é $\sqrt{4,605/80} = 0,240$, $\sqrt{4,605/15} = 0,554$ e $\sqrt{4,605/5} = 0,960$. Os escores ficam $a_1 : 0,860$, $a_2 : 1,104$, $a_3 : 1,660$ — a próxima simulação vai para a_3 , a ação *menos* visitada e de maior $\hat{Q}$. Já a *jogada real* seria a_1 , com 80 visitas: a regra de seleção é otimista de propósito, enquanto a regra de saída é conservadora de propósito. Para (c), precisamos de $0,70 + \sqrt{4,605/N} < 0,62 + 0,240 = 0,860$, isto é $\sqrt{4,605/N} < 0,160$, ou seja $N > 180$ — mais visitas do que o orçamento inteiro do exemplo. Ou seja: com $\hat{Q}$ superior, a_3 não perde a disputa por bônus; ela só perderia se seu $\hat{Q}$ caísse ao ser mais explorada — que é exatamente o que se espera que aconteça, e a razão de o método funcionar.

13.2. Para X_1, X_2 i.i.d. $\mathcal{N}(0, \sigma^2)$ vale $\mathbb{E}[\max(X_1, X_2)] = \sigma/\sqrt{\pi}$ (caso particular de $\mathbb{E}[\max] = \sigma \frac{n}{\sqrt{\pi}}$ para $n = 2$ braços, obtido de $\max(x, y) = \frac{x+y}{2} + \frac{|x-y|}{2}$ e $\mathbb{E}|X_1 - X_2| = 2\sigma/\sqrt{\pi}$). Como $\max_a \mathbb{E}[\hat{Q}(a)] = 0$, o excesso é puro artefato do ruído. Na busca de Monte Carlo simples, σ é o erro-padrão da média de K retornos, que cresce com a variância do retorno — e a variância do retorno cresce com o horizonte. Logo o viés otimista é maior justamente onde a busca é mais necessária. É esse mecanismo que motiva estimadores desacoplados (*double Q-learning*) e a preferência por contagem de visitas na saída do MCTS.

9 Glossário

Inglês	Português	Inglês	Português
simulation-based search	busca baseada em simulação	branching factor	fator de ramificação
decision-time planning	planejamento em tempo de decisão	anytime algorithm	algoritmo <i>anytime</i>
forward search	busca dirigida para frente	best-first search	busca best-first
expectimax tree	árvore expectimax	progressive widening	alargamento progressivo
rollout	<i>rollout</i> , simulação até o fim	self-play	auto-jogo
tree policy	política de árvore	policy prior	<i>prior</i> de política
selection / expansion	seleção / expansão	policy improvement	melhoria de política
backup	retropropagação	expert iteration	iteração por especialista
visit count	contagem de visitas	latent state	estado latente
upper confidence bound	limite superior de confiança	test-time compute	computação em tempo de decisão

Atenção

Erros comuns. Devolver $\arg \max \hat{Q}$ em vez de $\arg \max N$; esquecer de inverter o sinal do retorno em jogos de dois jogadores; usar c sem normalizar os retornos; descartar a árvore a cada jogada; treinar p contra o $\arg \max$ da busca em vez da distribuição de visitas; supor que um modelo com 1% de erro por passo serve para um horizonte de 50 passos.

10 Para ir além

- ▶ L. Kocsis e C. Szepesvári. Bandit based Monte-Carlo planning. *ECML*, 2006. — o artigo original do UCT.
- ▶ C. Browne et al. A survey of Monte Carlo tree search methods. *IEEE Transactions on CIAIG*, 4(1):1–43, 2012.
- ▶ P.-A. Coquelin e R. Munos. Bandit algorithms for tree search. *UAI*, 2007. — os limites de pior caso do UCT.
- ▶ D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.
- ▶ D. Silver et al. Mastering the game of Go without human knowledge. *Nature*, 550:354–359, 2017.
- ▶ D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362:1140–1144, 2018.
- ▶ T. Anthony, Z. Tian e D. Barber. Thinking fast and slow with deep learning and tree search. *NeurIPS*, 2017.
- ▶ J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.
- ▶ R. Sutton e A. Barto. *Reinforcement Learning: An Introduction*, 2ª ed., MIT Press, 2018. Capítulo 8.
- ▶ E. Brunskill. *CS234: Reinforcement Learning*, Stanford University — *Lecture 13: Monte Carlo Tree Search*, com material derivado de David Silver.